

Chapter 5 Exercises

5.11 Of these two types of programs:

- a. I/O-bound
- b. CPU-bound

which is more likely to have voluntary context switches, and which is more likely to have nonvoluntary context switches? Explain your answer.

5.12 Discuss how the following pairs of scheduling criteria conflict in certain settings.

- a. CPU utilization and response time
- b. Average turnaround time and maximum waiting time
- c. I/O device utilization and CPU utilization

5.13 One technique for implementing **lottery scheduling** works by assigning processes lottery tickets, which are used for allocating CPU time. Whenever a scheduling decision has to be made, a lottery ticket is chosen at random, and the process holding that ticket gets the CPU. The BTV operating system implements lottery scheduling by holding a lottery 50 times each second, with each lottery winner getting 20 milliseconds of CPU time ($20 \text{ milliseconds} \times 50 = 1 \text{ second}$). Describe how the BTV scheduler can ensure that higher-priority threads receive more attention from the CPU than lower-priority threads.

5.14 Most scheduling algorithms maintain a **run queue**, which lists processes eligible to run on a processor. On multicore systems, there are two general options: (1) each processing core has its own run queue, or (2) a single run queue is shared by all processing cores. What are the advantages and disadvantages of each of these approaches?

5.15 Consider the exponential average formula used to predict the length of the next CPU burst. What are the implications of assigning the following values to the parameters used by the algorithm?

- a. $\alpha = 0$ and $\tau_0 = 100$ milliseconds
- b. $\alpha = 0.99$ and $\tau_0 = 10$ milliseconds

5.16 A variation of the round-robin scheduler is the **regressive round-robin** scheduler. This scheduler assigns each process a time quantum and a priority. The initial value of a time quantum is 50 milliseconds. However, every time a process has been allocated the CPU and uses its entire time quantum (does not block for I/O), 10 milliseconds is added to its time quantum, and its priority level is boosted. (The time quantum for a process can be increased to a maximum of 100 milliseconds.) When a process blocks before using its entire time quantum, its time quantum is reduced by 5 milliseconds, but its priority remains the same. What type of process (CPU-bound or I/O-bound) does the regressive round-robin scheduler favor? Explain.

- 5.17 Consider the following set of processes, with the length of the CPU burst given in milliseconds:

<u>Process</u>	<u>Burst Time</u>	<u>Priority</u>
P_1	5	4
P_2	3	1
P_3	1	2
P_4	7	2
P_5	4	3

The processes are assumed to have arrived in the order P_1, P_2, P_3, P_4, P_5 , all at time 0.

- Draw four Gantt charts that illustrate the execution of these processes using the following scheduling algorithms: FCFS, SJF, non-preemptive priority (a larger priority number implies a higher priority), and RR (quantum = 2).
 - What is the turnaround time of each process for each of the scheduling algorithms in part a?
 - What is the waiting time of each process for each of these scheduling algorithms?
 - Which of the algorithms results in the minimum average waiting time (over all processes)?
- 5.18 The following processes are being scheduled using a preemptive, priority-based, round-robin scheduling algorithm.

<u>Process</u>	<u>Priority</u>	<u>Burst</u>	<u>Arrival</u>
P_1	8	15	0
P_2	3	20	0
P_3	4	20	20
P_4	4	20	25
P_5	5	5	45
P_6	5	15	55

Each process is assigned a numerical priority, with a higher number indicating a higher relative priority. The scheduler will execute the highest-priority process. For processes with the same priority, a round-robin scheduler will be used with a time quantum of 10 units. If a process is preempted by a higher-priority process, the preempted process is placed at the end of the queue.

- Show the scheduling order of the processes using a Gantt chart.
 - What is the turnaround time for each process?
 - What is the waiting time for each process?
- 5.19 The nice command is used to set the nice value of a process on Linux, as well as on other UNIX systems. Explain why some systems may allow any user to assign a process a nice value ≥ 0 yet allow only the root (or administrator) user to assign nice values < 0 .

- 5.20 Which of the following scheduling algorithms could result in starvation?
- First-come, first-served
 - Shortest job first
 - Round robin
 - Priority
- 5.21 Consider a variant of the RR scheduling algorithm in which the entries in the ready queue are pointers to the PCBs.
- What would be the effect of putting two pointers to the same process in the ready queue?
 - What would be two major advantages and two disadvantages of this scheme?
 - How would you modify the basic RR algorithm to achieve the same effect without the duplicate pointers?
- 5.22 Consider a system running ten I/O-bound tasks and one CPU-bound task. Assume that the I/O-bound tasks issue an I/O operation once for every millisecond of CPU computing and that each I/O operation takes 10 milliseconds to complete. Also assume that the context-switching overhead is 0.1 millisecond and that all processes are long-running tasks. Describe the CPU utilization for a round-robin scheduler when:
- The time quantum is 1 millisecond
 - The time quantum is 10 milliseconds
- 5.23 Consider a system implementing multilevel queue scheduling. What strategy can a computer user employ to maximize the amount of CPU time allocated to the user's process?
- 5.24 Consider a preemptive priority scheduling algorithm based on dynamically changing priorities. Larger priority numbers imply higher priority. When a process is waiting for the CPU (in the ready queue, but not running), its priority changes at a rate α . When it is running, its priority changes at a rate β . All processes are given a priority of 0 when they enter the ready queue. The parameters α and β can be set to give many different scheduling algorithms.
- What is the algorithm that results from $\beta > \alpha > 0$?
 - What is the algorithm that results from $\alpha < \beta < 0$?
- 5.25 Explain how the following scheduling algorithms discriminate either in favor of or against short processes:
- FCFS
 - RR
 - Multilevel feedback queues

- 5.26 Describe why a shared ready queue might suffer from performance problems in an SMP environment.
- 5.27 Consider a load-balancing algorithm that ensures that each queue has approximately the same number of threads, independent of priority. How effectively would a priority-based scheduling algorithm handle this situation if one run queue had all high-priority threads and a second queue had all low-priority threads?
- 5.28 Assume that an SMP system has private, per-processor run queues. When a new process is created, it can be placed in either the same queue as the parent process or a separate queue.
- What are the benefits of placing the new process in the same queue as its parent?
 - What are the benefits of placing the new process in a different queue?
- 5.29 Assume that a thread has blocked for network I/O and is eligible to run again. Describe why a NUMA-aware scheduling algorithm should reschedule the thread on the same CPU on which it previously ran.
- 5.30 Using the Windows scheduling algorithm, determine the numeric priority of each of the following threads.
- A thread in the `REALTIME_PRIORITY_CLASS` with a relative priority of `NORMAL`
 - A thread in the `ABOVE_NORMAL_PRIORITY_CLASS` with a relative priority of `HIGHEST`
 - A thread in the `BELOW_NORMAL_PRIORITY_CLASS` with a relative priority of `ABOVE_NORMAL`
- 5.31 Assuming that no threads belong to the `REALTIME_PRIORITY_CLASS` and that none may be assigned a `TIME_CRITICAL` priority, what combination of priority class and priority corresponds to the highest possible relative priority in Windows scheduling?
- 5.32 Consider the scheduling algorithm in the Solaris operating system for time-sharing threads.
- What is the time quantum (in milliseconds) for a thread with priority 15? With priority 40?
 - Assume that a thread with priority 50 has used its entire time quantum without blocking. What new priority will the scheduler assign this thread?
 - Assume that a thread with priority 20 blocks for I/O before its time quantum has expired. What new priority will the scheduler assign this thread?

- 5.33 Assume that two tasks, A and B , are running on a Linux system. The nice values of A and B are -5 and $+5$, respectively. Using the CFS scheduler as a guide, describe how the respective values of `vruntime` vary between the two processes given each of the following scenarios:
- Both A and B are CPU-bound.
 - A is I/O-bound, and B is CPU-bound.
 - A is CPU-bound, and B is I/O-bound.
- 5.34 Provide a specific circumstance that illustrates where rate-monotonic scheduling is inferior to earliest-deadline-first scheduling in meeting real-time process deadlines?
- 5.35 Consider two processes, P_1 and P_2 , where $p_1 = 50$, $t_1 = 25$, $p_2 = 75$, and $t_2 = 30$.
- Can these two processes be scheduled using rate-monotonic scheduling? Illustrate your answer using a Gantt chart such as the ones in Figure 5.21–Figure 5.24.
 - Illustrate the scheduling of these two processes using earliest-deadline-first (EDF) scheduling.
- 5.36 Explain why interrupt and dispatch latency times must be bounded in a hard real-time system.
- 5.37 Describe the advantages of using heterogeneous multiprocessing in a mobile system.